AI-ML for Mechanical Engineers
Yogesh Haribhau Kulkarni

Session 3: EDA DataPrep
Understanding Your Data

Why Look at Data Before Modeling?

- Datasets typically used in academic settins are clean and curated, but real datasets arrive messy: missing values, wrong types, outliers, duplicate rows.
- A model trained on unexamined data doesn't fail loudly, it just quietly learns the wrong thing, or learns noise instead of signal.
- "Garbage in, garbage out", no algorithm, however sophisticated, can fix what bad input data already broke.

A Concrete Example: Predicting Rain

Say the goal is a simple model: given today's readings, will it rain tomorrow?

Pressure (hPa)	Temperature (°C)	Humidity (%)	Rain
1013	28	65	No
1009	26	78	Yes
1018	31	40	No
1005	24	85	Yes
1015	29	55	No
1002	22	90	Yes

A dataset like this, clean, consistent, nothing missing, is what most textbook examples show you. Real data almost never looks like this.

The Same Data, As It Usually Arrives

Pressure (hPa)	Temperature (°C)	Humidity (%)	Rain
1013	28	65	No
1009	26	–	Yes
1018	310	40	No
1005	24	85	yes
1015	29	55	No
1015	29	55	No
1002	22	90	Yes

- Row 2: humidity is **missing**.
- Row 3: a temperature of 310°C is an **outlier**, a sensor glitch or typo, not a real reading.
- Row 4: "yes" breaks the **consistent labeling** used everywhere else ("Yes"/"No").
- Rows 5 and 6 are an exact **duplicate**.

Every one of these would silently distort a model trained without first looking.

What is Exploratory Data Analysis (EDA)?

- EDA is the practice of looking at a dataset, through summaries, tables, and plots, *before* building any model, to understand what you actually have.
- It answers questions like: What does each column mean? What range of values is normal? Are any values missing or clearly wrong? Which variables seem related to each other?
- EDA is detective work, not decoration, every chart or table should be answering a specific question about the data, not just filling a slide.

Two Angles: Univariate and Bivariate

- **Univariate analysis** looks at one column at a time: What's its distribution? Its mean, median, spread? Is it skewed? Does it have outliers?
- **Bivariate / multivariate analysis** looks at relationships *between* columns: Do two numeric columns move together (correlation)? Does a categorical column split a numeric one into different groups?

What EDA Is Looking For

- **Missing data**, which columns have gaps, and are they random or systematic?
- **Outliers**, values far outside the normal range; sometimes real, sometimes sensor glitches or data-entry errors.
- **Wrong types**, a numeric column read in as text, a date stored as a string.
- **Imbalance**, for a target column, are the classes roughly even, or is one outcome rare (as with most failure/fraud/churn-style targets)?

Modern EDA: Automated Profiling

- Things like missing data, outliers, types, imbalance, can be generated automatically as a first-pass report, before writing any manual pandas code.
- Tools like `ydata-profiling` (formerly pandas-profiling) and `sweetviz` scan a DataFrame and produce a full HTML report: per-column distributions, correlations, missing-value maps, warnings about high cardinality or skew.
- Useful as a fast starting point, not a replacement for judgment, it flags what to look at, it doesn't decide what matters for your specific problem.

```python
from ydata_profiling import ProfileReport

# df: assumed already loaded, e.g. df =
    pd.read_csv("telecom_churn.csv")
profile = ProfileReport(df, title="Churn Dataset
    Report")
profile.to_file("report.html")
```

Recap: Why EDA Matters

- Real data arrives messy: EDA is how you find out *how* messy, before that cost gets baked into a trained model.
- Two angles: **univariate** (one column at a time) and **bivariate** (relationships between columns).
- What to look for: missing data, outliers, wrong types, and class imbalance.
- Automated profiling tools give a fast first pass; they flag issues, they don't decide what matters for your problem.

From Exploring to Preparing

- Once EDA surfaces a problem, missing values, a skewed feature, an unencoded category , **data preparation** is the set of techniques used to fix it before modeling.
- The two are a loop, not a strict sequence: you explore, prepare, then often explore again to confirm the fix worked.
- The toolkit: handling missing values, scaling, encoding, and engineering new features, then closes with one dataset walked through start to finish.

Data Preprocessing

- Data Exploration phase results in finding data quality issues. Outliers, missing values, etc
- Feature Engineering is the way to prepare data.

Feature Engineering

The Art of Feature Engineering

What is Feature Engineering?

- The science (and art) of extracting more information from existing data.
- E.g. To predict footfall in a mall, if you collect data date wise, it may not have meaningful insight, but if you collect day-of-the-week wise, may get some patterns.
- This is done as part of Data Preparation, before doing Machine Learning modeling.

Feature Engineering: The Process

What is the process of Feature Engineering ?

- Variable transformation.
- Variable / Feature creation.

Original Column	Transformation	Creation
Income (skewed)	log(Income)	–
Order Date	–	Day-of-week, Is-weekend

Data Processing: Variable Transformations

PreProcessing Techniques

Changing the way data is represented just to make it more compatible with certain machine learning algorithms:

- Scaling: Normalization, Standardization
- Missing value replacement
- Binning
- Sampling
- Feature Selection
- Dimensionality Reduction: PCA
- Encoding

Scaling

Normalization

Typical ranges used for normalizing feature values are [0,1] and [-1,1].

- Change a continuous feature to fall within a specified range while maintaining the relative differences between the values for the feature
- Example: Customer ages in a data-set: [16, 96] Customer salaries: [10000, 100000]
- Range Normalization: convert a feature value into the range [low , high]
- Re-parametrization to the new range.

Sensitive to the presence of Outliers in a data-set.

Sklearn Normalization

- Use function sklearn.preprocessing.normalize()
- X: Data to be normalized
- norm: which norm to use: l1 (sum of elements gives 1) or l2 (sqrt the sum of the squares of the elements gives one)
- axis: whether to normalize by row or column
- In practice, prefer the class-based `Normalizer` (fit once, reuse via transform), covered hands-on in Session 7

```
from sklearn import preprocessing
import numpy as np

X = np.array([[ 1., -1.,  2.],
              [ 2.,  0.,  0.],
              [ 0.,  1., -1.]])

X_normalized = preprocessing.normalize(X,
    norm='l2')
print(X_normalized)

array([[ 0.408, -0.408,  0.816],
       [ 1.   ,  0.   ,  0.   ],
       [ 0.   ,  0.707, -0.707]])
```

Standardization

Standardization: To transform data so that it has zero mean and unit variance. Also called scaling. Done by removing the mean value from all points, then scale it by dividing with the standard deviation.

- Use function sklearn.preprocessing.scale()
- X: Data to be scaled.
- with_mean: Boolean. Make zero mean or not.
- with_std: Make unit standard deviation or not.

Standardization: Z-Score

Majority of feature values will be in a range of [-1, 1].

- Measures how many standard deviations a feature value is from the mean for that feature
- Mean = 0, Standard deviation = 1
- $StandardScores = a'_i = \frac{a_i - \bar{a}}{sd(a)}$

Standardization assumes the feature values are normally distributed. If not, standardization may introduce some distortions.

Standardization Example

	Height			Sponsorship Earnings		
	Values	Range	Standard	Values	Range	Standard
	192	0.500	-0.073	561	0.315	-0.649
	197	0.679	0.533	1,312	0.776	0.762
	192	0.500	-0.073	1,359	0.804	0.850
	182	0.143	-1.283	1,678	1.000	1.449
	206	1.000	1.622	314	0.164	-1.114
	192	0.500	-0.073	427	0.233	-0.901
	190	0.429	-0.315	1,179	0.694	0.512
	178	0.000	-1.767	1,078	0.632	0.322
	196	0.643	0.412	47	0.000	-1.615
	201	0.821	1.017	1111	0.652	0.384
Max	206			1,678		
Min	178			47		
Mean	193			907		
Std Dev	8.26			532.18		

Sklearn Standardization/Scaling

This function-based form is handy for a quick one-off transform; for a real pipeline (fit on train, reuse on test) use the class-based `StandardScaler`, covered hands-on in Session 7.

```
from sklearn import preprocessing
import numpy as np

X = np.array([[ 1., -1.,  2.],
              [ 2.,  0.,  0.],
              [ 0.,  1., -1.]])

X_scaled = preprocessing.scale(X)

print(X_scaled)

array([[ 0.  ..., -1.22...,  1.33...],
       [ 1.22...,  0.  ..., -0.26...],
       [-1.22...,  1.22..., -1.06...]])
```

Binning

Binning

- Converting a continuous feature into a categorical feature
- Define a series of ranges (called bins) for the continuous feature that correspond to the levels of the new categorical feature
- Approaches:
  - equal-width binning
  - equal-frequency binning
- Need to "manually" decide the number of bins:
  - Choosing a low number may lose a lot of information
  - Choosing a very high number might result in a very few instances in each bin or empty bins

Equal-Width Binning

- Splits the range of the feature values into b bins each of size $\frac{range}{b}$
- Usually works well
- Some near-empty bins when data follows a normal distribution

Example: Ages = 12, 15, 18, 22, 25, 28, 32, 35, 40, 55 (range 12–55, 3 bins of width ≈14.3)

Bin	Range	Ages in Bin
1	[12, 26.3)	12, 15, 18, 22, 25
2	[26.3, 40.7)	28, 32, 35, 40
3	[40.7, 55]	55

Equal-Frequency Binning

- Algorithm:
  - Sorts the continuous feature values into ascending order
  - Then places an equal number of instances into each bin, starting with bin 1
- Number of instances placed in each bin $\frac{TotalNumberOfInstances}{NumberOfBins}$

Same ages, equal-frequency (3 bins, ≈3–4 per bin):

Bin	Ages in Bin	Range Covered
1	12, 15, 18	[12, 18]
2	22, 25, 28	[22, 28]
3	32, 35, 40, 55	[32, 55]

Same bin sizes, uneven range widths: the opposite trade-off from equal-width.

Data Preparation: Sampling

- Sometimes we have too much data!, instead sample a smaller percentage from the larger dataset
- Care required when sampling:
  - Try to ensure that the resulting datasets is still representative of the original data and that no unintended bias is introduced during this process.
  - If not, any modeling on the sample will not be relevant to the overall dataset.

```
# df: assumed already loaded, e.g. df =
    pd.read_csv("telecom_churn.csv")
# Plain random sample, risky if the target is
    imbalanced (e.g. 86/14 churn split):
# a small random sample could end up with too few
    (or zero) churners.
df.sample(frac=0.2, random_state=42)

# Stratified sample, keeps each class's proportion
    intact
df.groupby('Churn', group_keys=False).apply(
    lambda g: g.sample(frac=0.2, random_state=42))
```

Handling Outliers

Clamp Transformation

- Clamps all values above an upper threshold and below a lower threshold to remove outliers
- Upper and lower thresholds can be set manually based on domain knowledge
- Or:
  - Lower = 1st quartile -1.5 x inter-quartile range
  - Upper = 3rd quartile + 1.5 x inter-quartile range
- Lower and upper are the specific thresholds.

Case Study: Outliers

Data quality issues found in the motor Insurance fraud dataset

(a) Continuous Features

Feature	Count	% Miss.	Card.	Min	1st Qrt.	Mean	Median	3rd Qrt.	Max	Std. Dev.
Income	500	0.0	171	0.0	0.0	13,740.0	0.0	33,918.5	71,284.0	20,081.5
Num Claimants	500	0.0	4	1.0	1.0	1.9	2	3.0	4.0	1.0
Claim Amount	500	0.0	493	-99,999	3,322.3	16,373.2	5,663.0	12,245.5	270,200.0	29,426.3
Total Claimed	500	0.0	235	0.0	0.0	9,597.2	0.0	11,282.8	729,792.0	35,655.7
Num Claims	500	0.0	7	0.0	0.0	0.8	0.0	1.0	56.0	2.7
Num Soft Tissue	500	2.0	6	0.0	0.0	0.2	0.0	0.0	5.0	0.6
% Soft Tissue	500	0.0	9	0.0	0.0	0.2	0.0	0.0	2.0	0.4
Amount Received	500	0.0	329	0.0	0.0	13,051.9	3,253.5	8,191.8	295,303.0	30,547.2
Fraud Flag	500	0.0	2	0.0	0.0	0.3	0.0	1.0	1.0	0.5

(a) Categorical Features

Feature	Count	% Miss.	Card.	Mode	Mode Freq.	Mode %	2nd Mode	2nd Mode Freq.	2nd Mode %
Insurance Type	500	0.0	1	CI	500	1.0	–	–	–
Marital Status	500	61.2	4	Married	99	51.0	Single	48	24.7
Injury Type	500	0.0	4	Broken Limb	177	35.4	Soft Tissue	172	34.4
Hospital Stay	500	0.0	2	No	354	70.8	Yes	146	29.2

Handling Missing Value

Handling Missing Data

- Motivation: We will frequently encounter missing values, especially in big data. Lots of fields, lots of observations
- Question: how to handle the missing data?

How much data is missing?

- Handling Missing Data:
  - Dataset of 30 variables
  - 5% of data is missing
  - Missing values are spread evenly throughout data
- 80% of records would have at least one missing value
- Bad solution: Deleting records with any missing data

Handling Missing Values (Dropping)

- Drop any features that have missing values. Might lead to massive loss of data
- Drop any instance/record that has a missing value. Might lead to bias
- Derive a missing indicator feature from features with missing values. Replace with a binary feature, whether the value was missing or not.
- Ignore missing values. If data mining method is robust

Example: 4 records, `Age` missing in row 2, `Income` missing in row 3.

- Drop-column: removes `Age` *and* `Income` entirely (each has a gap somewhere), keeping all 4 rows but losing 2 features.
- Drop-row: removes rows 2 and 3, keeping both features but left with only 2 of 4 records.
- Indicator: keeps all 4 rows and both columns, adding `Age_missing` and `Income_missing` flags instead of discarding anything.

Handling Missing Values (Imputation)

- Replace missing value with some constant (Specified by analyst)
- Replace missing value with mean, median, or mode
- Replace missing value with value generated at random from the observed variable distribution
- Replace missing value with imputed values, based on other characteristics of the record

Data Frames: Missing Values

Missing values are marked as NaN

	year	month	day	dep_time	dep_delay	arr_time	arr_delay	carrier	tailnum	flight	origin	dest	air_time	distance	hour	minute
330	2013	1	1	1807.0	29.0	2251.0	NaN	UA	N31412	1228	EWR	SAN	NaN	2425	18.0	7.0
403	2013	1	1	NaN	NaN	NaN	NaN	AA	N3EHAA	791	LGA	DFW	NaN	1389	NaN	NaN
404	2013	1	1	NaN	NaN	NaN	NaN	AA	N3EVAA	1925	LGA	MIA	NaN	1096	NaN	NaN
855	2013	1	2	2145.0	16.0	NaN	NaN	UA	N12221	1299	EWR	RSW	NaN	1068	21.0	45.0
858	2013	1	2	NaN	NaN	NaN	NaN	AA	NaN	133	JFK	LAX	NaN	2475	NaN	NaN

```
# Read a dataset with missing values
flights = pd.read_csv(

    "http://rcs.bu.edu/examples/python/data_analysis/flights.csv")

# Select the rows that have at least one missing
    value
flights[flights.isnull().any(axis=1)].head()
```

Missing Value Replacement

In scikit-learn, this is referred to as "Imputation"

- Use class `sklearn.impute.SimpleImputer`
- `strategy`: What to replace the missing value with: `mean` / `median` / `most_frequent`
- `missing_values`: Value to treat as missing (e.g. `np.nan`)

Example code for Replacing Missing Values

```
import numpy as np
from sklearn.impute import SimpleImputer

X = np.array([[23.56], [53.45], [np.nan], [44.44],
    [77.78],
              [np.nan], [234.44], [11.33],
    [79.87]])

print(X)

imp = SimpleImputer(missing_values=np.nan,
    strategy='mean')
X = imp.fit_transform(X)

print(X)
```

Encoding

Encoding

- Scikit-learn doesn't directly handle categorical attributes well.
- In order to use them in the dataset, some sort of encoding needs to be performed.
- One good way to encode categorical attributes: if there are n categories, create n dummy binary variables representing each category.
- Can be done easily using the sklearn.preprocessing.OneHotEncoder class.

Encoding: Sample Data

```
raw_data = {'first_name': ['Jason', 'Molly',
    'Tina', 'Jake', 'Amy'],
        'last_name': ['Miller', 'Jacobson', 'Ali',
    'Milner', 'Cooze'],
        'age': [42, 52, 36, 24, 73],
        'city': ['San Francisco', 'Baltimore',
    'Miami', 'Douglas', 'Boston']}
df = pd.DataFrame(raw_data, columns=['first_name',
    'last_name', 'age', 'city'])
```

	first_name	last_name	age	city
0	Jason	Miller	42	San Francisco
1	Molly	Jacobson	52	Baltimore
2	Tina	Ali	36	Miami
3	Jake	Milner	24	Douglas
4	Amy	Cooze	73	Boston

Imp Note

- All pre-processing steps done on X of training have to be done on X of testing also.
- Once results are predicted, you may need to reverse transform some of the features to find corresponding points in the original X.

Handling Intra Dependence

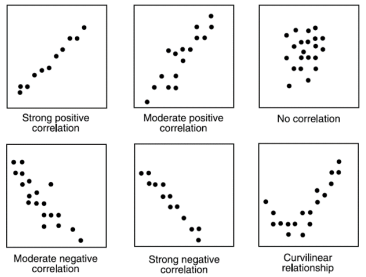
Covariance

- Visual preliminary exploration, comparing two variables, using scatter plots
- Quantitative preliminary exploration using covariance and correlation measures
- Covariance: $cov(a,b) = 1/(n-1)\sum((a_i - \bar{a}) \times (b_i - \bar{b}))$

Intuition

Covariance tells you the *direction* two features move together (positive/negative) but its magnitude is meaningless on its own: it's stuck in the product of the two features' original units, so you can't compare one pair's covariance to another's without normalizing. That's exactly the problem correlation (next slide) fixes.

Covariance: Visual Intuition



Covariance: A Worked Example

What is interesting in this data?

	Height		Weight		$(h-\bar{h})\times$	Age		$(h-\bar{h})\times$
ID	(h)	$h-\bar{h}$	(w)	$w-\bar{w}$	$(w-\bar{w})$	(a)	$a-\bar{a}$	$(a-\bar{a})$
1	192	0.9	218	3.0	2.7	29	2.6	2.3
2	218	26.9	251	36.0	967.5	35	8.6	231.3
3	197	5.9	221	6.0	35.2	22	-4.4	-26.0
4	192	0.9	219	4.0	3.6	22	-4.4	-4.0
5	198	6.9	223	8.0	55.0	29	2.6	17.9
				...				
26	191	-0.1	218	3.0	-0.3	19	-7.4	0.7
27	196	4.9	235	20.0	97.8	32	5.6	27.4
28	198	6.9	221	6.0	41.2	22	-4.4	-30.4
29	207	15.9	247	32.0	508.3	27	0.6	9.5
30	201	9.9	244	29.0	286.8	25	-1.4	-13.9
Mean	191.1		215.0			26.4		
Std Dev	13.6		19.8			4.2		
Sum					7,009.9			570.8

$$cov(\text{HEIGHT}, \text{WEIGHT}) = \frac{7{,}009.9}{29} = 241.72$$
$$cov(\text{HEIGHT}, \text{AGE}) = \frac{570.8}{29} = 19.7$$

Correlation: Visual Intuition

	Height		Weight		$(h-\bar{h})\times$	Age		$(h-\bar{h})\times$
ID	(h)	$h-\bar{h}$	(w)	$w-\bar{w}$	$(w-\bar{w})$	(a)	$a-\bar{a}$	$(a-\bar{a})$
1	192	0.9	218	3.0	2.7	29	2.6	2.3
2	218	26.9	251	36.0	967.5	35	8.6	231.3
3	197	5.9	221	6.0	35.2	22	-4.4	-26.0
4	192	0.9	219	4.0	3.6	22	-4.4	-4.0
5	198	6.9	223	8.0	55.0	29	2.6	17.9
				...				
26	191	-0.1	218	3.0	-0.3	19	-7.4	0.7
27	196	4.9	235	20.0	97.8	32	5.6	27.4
28	198	6.9	221	6.0	41.2	22	-4.4	-30.4
29	207	15.9	247	32.0	508.3	27	0.6	9.5
30	201	9.9	244	29.0	286.8	25	-1.4	-13.9
Mean	191.1		215.0			26.4		
Std Dev	13.6		19.8			4.2		
Sum					7,009.9			570.8

$$corr(Height, Weight) = \frac{241.72}{13.6 \times 19.8} = 0.898$$
$$corr(Height, Age) = \frac{19.7}{13.6 \times 4.2} = 0.345$$

Correlation

- Normalized form of covariance
- Values ranges between -1 and +1
- Correlation: $corr(a,b) = \frac{cov(a,b)}{sd(a) \times sd(b)}$

- values fall into the range [-1, 1]
  - values close to -1 indicate a very strong negative correlation (or covariance)
  - values close to 1 indicate a very strong positive correlation
  - values around 0 indicate no correlation
- Features that have no correlation are said to be independent.
- Calculating correlation between the HEIGHT feature and the WEIGHT and AGE features from the basketball players dataset.

Covariance and Correlation Matrix

- There are usually multiple continuous features in a dataset to explore.
- Covariance and Correlation Matrix for the display of every combination of features

$$\sum_{\{a,b,\ldots,z\}} = \begin{bmatrix} var(a) & cov(a,b) & \cdots & cov(a,z) \\ cov(b,a) & var(b) & \cdots & cov(b,z) \\ \vdots & \vdots & \ddots & \vdots \\ cov(z,a) & cov(z,b) & \cdots & var(z) \end{bmatrix} \quad \text{correlation matrix}_{\{a,b,\ldots,z\}} = \begin{bmatrix} corr(a,a) & corr(a,b) & \cdots & corr(a,z) \\ corr(b,a) & corr(b,b) & \cdots & corr(b,z) \\ \vdots & \vdots & \ddots & \vdots \\ corr(z,a) & corr(z,b) & \cdots & corr(z,z) \end{bmatrix}$$

Covariance/Correlation Matrix Example

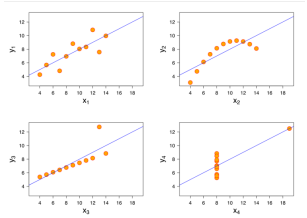
ID	Height	$h-\bar{h}$	Weight	$w-\bar{w}$	$(h-\bar{h})\times(w-\bar{w})$	Age	$a-\bar{a}$	$(h-\bar{h})\times(a-\bar{a})$
1	192	0.9	218	3.0	2.7	29	2.6	2.3
2	218	26.9	251	36.0	967.5	35	8.6	231.3
3	197	5.9	221	6.0	35.2	22	-4.4	-26.0
4	192	0.9	219	4.0	3.6	22	-4.4	-4.0
5	198	6.9	223	8.0	55.0	29	2.6	17.9
...								
26	191	-0.1	218	3.0	-0.3	19	-7.4	0.7
27	196	4.9	235	20.0	97.8	32	5.6	27.4
28	198	6.9	221	6.0	41.2	22	-4.4	-30.4
29	207	15.9	247	32.0	508.3	27	0.6	9.5
30	201	9.9	244	29.0	286.8	25	-1.4	-13.9
Mean	191.1		215.0			26.4		
Std Dev	13.6		19.8			4.2		
Sum					7,009.9			570.8

$$\sum_{\text{<Height,Weight,Age>}} = \begin{bmatrix} 185.128 & 241.72 & 19.7 \\ 241.72 & 392.102 & 24.469 \\ 19.7 & 24.469 & 17.697 \end{bmatrix}$$

$$\text{correlation matrix}_{\text{<Height,Weight,Age>}} = \begin{bmatrix} 1.0 & 0.898 & 0.345 \\ 0.898 & 1.0 & 0.294 \\ 0.345 & 0.294 & 1.0 \end{bmatrix}$$

Correlation does not Imply Causation

$$\text{correlation matrix}_{\text{<Height,Weight,Age>}} = \begin{bmatrix} 1.0 & 0.898 & 0.345 \\ 0.898 & 1.0 & 0.294 \\ 0.345 & 0.294 & 1.0 \end{bmatrix}$$



- Correlation is a good measure of the relationship between two continuous features, but it is not by any means perfect.
- Still need visual analysis.

Data Processing: Variable Creation/Reduction

Feature Creation & Its Benefits

What is Feature / Variable Creation & its Benefits?

- Generate a new variables / features based on existing variable(s).
- For example, say, we have date(dd-mm-yy) as an input variable in a data set. We can generate new variables like day, month, year, week, weekday that may have better relationship with target variable.

Emp_Code	Gender	Date	New_Day	New_Month	New_Year
A001	Male	21-Sep-11	21	9	2011
A002	Female	27-Feb-13	27	2	2013
A003	Female	14-Nov-12	14	11	2012
A004	Male	07-Apr-13	7	4	2013
A005	Female	21-Jan-11	21	1	2011
A006	Male	26-Apr-13	26	4	2013
A007	Male	15-Mar-12	15	3	2012

Dealing with High Dimensionality

- Feature Selection
- Feature Extraction
- Data Engineering: Processing, Assumptions
- Tree-based Approaches
- Factorization-based Approaches
- Neural Network based Approaches

Feature selection

- Use variables giving good prediction (information gain)
- Use domain knowledge to filter dependent features

Feature extraction

- Using ALL current features, combine them into smaller set.
- Combinations of variables may be more effective bases for insights, even if physical meaning is obscure
- Some combine linear others non-linearly

Aggregation

- Combining two or more attributes into a single attribute Or - combining two or more objects into a single object
- Purpose:
  - Data reduction: reduce # of attributes/objects
  - Change of scale: high-level vs. low-level
- Motivations: Less memory and processing time

Example: daily transactions → monthly summary.

Date	Sales
2024-01-01	120
2024-01-02	95
⋮	⋮
2024-01-31	110

→

Month	Total Sales
2024-01	3,450

31 daily rows collapse into 1 monthly row: less memory, coarser (but often sufficient) granularity.

Other Issues

- Inconsistent Values Example:
  - Data object with address, city, zip code in three separate fields
  - But address / city is in a different zip code
- Some inconsistencies are easy to detect (and fix) automatically; others are not.
- Duplicate Data Example:
  - many people receive duplicate mailings because they are in a database multiple times under slightly different names

When to Remove Variables

- Variables that will not help the analysis should be removed:
  - Unary variables: take on a single value Example: gender variable for students at an all-girls school
  - Variables that are nearly unary Example: gender of football athletes at elementary school 99.95% of the players are male
  - Some data mining algorithms may treat the variable as unary Not enough data to investigate the female players anyway
- Think carefully before removing variables because of:
  - 90% of the values are missing
  - Strong correlation between two variables

When to Remove Variables: Mostly-Missing Case

90% of the values are missing:

- Are the values that are present representative or not?
- If the present values are representative, then either (1) remove the variable or (2) impute the values.
- If the present values are non-representative, their presence adds value.
  - Scenario: donation_dollars field in a self-reported survey
  - Assumption: those who donate a lot are more inclined to report their donation
- Could also binarize the variable: donation_flag

ML-based Approaches

- Decision Tree/Random Forest
- Gives most important features

```
from sklearn.ensemble import RandomForestClassifier

# X_train, y_train, feature_names: assumed already
    prepared, e.g. after a
# train/test split on the cleaned, encoded
    DataFrame
model = RandomForestClassifier()
model.fit(X_train, y_train)

# Pair each feature name with its importance, sort
    descending
importances =
    sorted(zip(model.feature_importances_,
    feature_names), reverse=True)
print("Features sorted by their score:")
print(importances)

# Output:
# [(0.2321, 'fractal_dimension_mean'), (0.1536,
    'concavity_worst'),
#  (0.1164, 'radius_mean'), (0.0889,
    'concave_points_sd_error'), ...]
```

Recap: The Data Preparation Toolkit

- **Scaling**: normalization and standardization, so features on different units and ranges become comparable.
- **Missing values**: drop, flag, or impute (constant, mean/median/mode, or model-based).
- **Binning**: turn a continuous feature into categories, equal-width or equal-frequency.
- **Encoding**: turn categories into numbers models can use (one-hot, etc.).
- **Feature creation & reduction**: engineer new columns, or cut down to the ones that actually carry signal.

Same discipline as before: whatever preprocessing you fit on training data must be applied the same way to test data. Next, we walk one real dataset through this entire loop, start to finish.

End-to-End Example: Customer Churn

The Question

A telecom company wants to know: which customers are likely to *churn* (cancel their service)? We'll walk one dataset through the full arc: load, assess, describe, visualize, engineer features, check what matters. This is the same pattern you'll reuse on almost every dataset in this course.

- Dataset: per-customer usage records (call minutes, charges, plan type) plus whether that customer eventually churned.

Step 1: Load Data

```
import pandas as pd

df = pd.read_csv('telecom_churn.csv')
df.shape
(3333, 20)
```

Step 2: First Look at the Data

Before anything structured, just look at a few rows (shown here for 4 of the 20 columns; the rest are just as real, only trimmed to fit the slide).

```
df.head(3)
  State  Account length  Total day minutes  Churn
0    KS             128              265.1  False
1    OH             107              161.6  False
2    NJ             137              243.4  False
```

Step 3: Initial Assessment

Before anything else: what columns exist, and what type is each one?

```
df.columns
Index(['State', 'Account length', 'Area code',
    'International plan',
    'Voice mail plan', 'Total day minutes',
    'Total day calls',
    'Total eve minutes', 'Customer service
    calls', 'Churn', ...])

df.dtypes
State                  object
International plan     object
Total day minutes     float64
Customer service calls  int64
Churn                    bool
dtype: object
```

Step 4: Descriptive Statistics

`describe()` gives a fast univariate read on every numeric column at once: range, center, spread.

```
df.describe()
       Account length  Total day minutes  Customer
    service calls
count         3333.00            3333.00
           3333.00
mean           101.06             179.78
               1.56
std             39.82              54.47
               1.32
min              1.00               0.00
               0.00
max             243.00             350.80
               9.00
```

Step 5: Data Quality Check

Intuition

No missing values or duplicates here, but the churn split (about 86/14) is a real finding, not a non-issue: with churners this rare, a model that just always predicts "stays" would already be 85% "accurate" while being completely useless: a pattern to watch for once we get to evaluation metrics.

```
df.isnull().sum().sum()
0

df.duplicated().sum()
0

df['Churn'].value_counts(normalize=True)
False    0.855
True     0.145
Name: Churn, dtype: float64
```

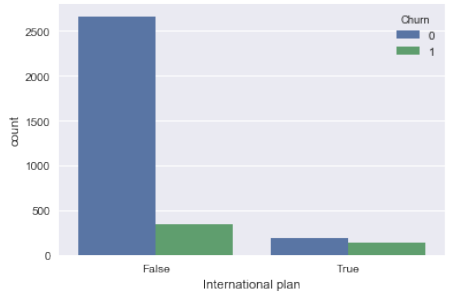
Step 6: Visualize, Distributions

Plotting a numeric column shows shape that summary statistics alone can hide (skew, multiple peaks, a long tail of outliers).

International plan	False	True	All
Churn			
0	2664	186	2850
1	346	137	483
All	3010	323	3333

Step 6: Visualize, Grouped by Target

Splitting a distribution by the target column is where visualization turns into real signal: do churners and non-churners actually look different?



Step 7: Feature Engineering, New Columns

Raw columns rarely capture the most useful signal directly: combine them into something more meaningful.

- `Total minutes` collapses three columns into one overall usage signal.
- `High service calls` turns a raw count into a flag: customers who call support a lot are often frustrated customers.

```
df['Total minutes'] = (df['Total day minutes'] +
    df['Total eve minutes']
                       + df['Total night
    minutes'])
```

```
df['High service calls'] = df['Customer service
    calls'] >= 4
```

Step 7: Feature Engineering, Encoding

`International plan` and `Voice mail plan` are Yes/No text: most models need numbers.

```
df['International plan'] = (df['International
    plan'] == 'Yes').astype(int)
df['Voice mail plan'] = (df['Voice mail plan'] ==
    'Yes').astype(int)

df[['International plan', 'Voice mail
    plan']].head(3)
   International plan  Voice mail plan
0                   0                1
1                   0                1
2                   0                0
```

Step 8: What Actually Matters? Correlation

Intuition

None of these correlations is huge on its own: that's normal and expected. It's a first pass, not a final answer: it tells you where to look closer (service calls, day-time usage, having an international plan) before committing to a full model in the next session.

```
df.corr(numeric_only=True)['Churn'].sort_values(
    ascending=False)
Customer service calls    0.21
High service calls        0.19
Total day minutes         0.21
International plan        0.26
Voice mail plan          -0.10
```

Step 8: What Actually Matters? Group Comparison

Correlation only captures straight-line relationships: groupby often reveals more. Here, churners call support noticeably more on average, exactly the kind of pattern a model can learn from.

```
df.groupby('Churn')['Customer service
    calls'].mean()
Churn
False    1.45
True     2.23
Name: Customer service calls, dtype: float64
```

Recap: The Pattern

- **Load**: get the data into a DataFrame.
- **Assess**: check shape, columns, types, and preview a few rows.
- **Describe**: summary statistics, missing values, duplicates, class balance.
- **Visualize**: distributions, and distributions split by the target.
- **Engineer**: new columns, encode categoricals.
- **Check what matters**: correlation and groupby comparisons.

This exact loop is what the rest of this course's ML sessions assume you've already done before a single model gets trained.

Quick Check: EDA & Data Preparation

Think About It

The churn split was about 86% stay / 14% churn. Why does that number matter more for choosing an *evaluation metric* later than it does for the EDA and preparation steps we just did?

Answer

Nothing in loading, cleaning, or engineering features depends on the class balance: those steps work the same either way. But once you train a model, plain accuracy becomes misleading with an 86/14 split, since always predicting "stays" already scores 86%. EDA's job here is to *surface* the imbalance early; deciding what to do about it (a different metric, resampling, etc.) is a modeling-stage decision, covered later in this course.